

ASSIGNMENT – 5

Binary Classification using Naive Bayes and K-Nearest Neighbors

AIM

To implement Naive Bayes and K-Nearest Neighbors (KNN) classifiers for a binary classification problem, evaluate them using multiple performance metrics, visualize model behavior, and analyze overfitting, underfitting, and bias-variance characteristics.

CODE

```
import os
import time
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neighbors import KDTree, BallTree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_curve, roc_auc_score

# -----
# 1. Load the Spambase dataset (binary classification)
# -----
csv_path = r'C:/Users/srik2/Desktop/College/Machine Learning/exp5/spambase.data'
# Spambase has no header - last column is label (1=spam, 0=not spam)
df = pd.read_csv(csv_path, header=None)
X = df.iloc[:, :-1].values # all columns except last = features
y = df.iloc[:, -1].values # last column = label

print(f"Dataset: {X.shape[0]} samples, {X.shape[1]} features")
print(f"Class distribution: {np.sum(y==0)} not-spam, {np.sum(y==1)} spam")

# Split into train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features (important for KNN distance calculations)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# -----
# 2. Train Naive Bayes variants
# -----
print("\n--- Naive Bayes ---")
nb_models = {
    "GaussianNB": GaussianNB(),
    "MultinomialNB": MultinomialNB(),
    "BernoulliNB": BernoulliNB()
```

```
}

nb_results = {}
for name, model in nb_models.items():
    t0 = time.time()
    model.fit(X_train, y_train) # NB doesn't need scaling
    train_time = time.time() - t0

    t0 = time.time()
    y_pred = model.predict(X_test)
    pred_time = time.time() - t0

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # Specificity = TN / (TN + FP)
    tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
    spec = tn / (tn + fp) if (tn + fp) > 0 else 0
    fpr = fp / (fp + tn) if (fp + tn) > 0 else 0

    nb_results[name] = {"acc": acc, "prec": prec, "rec": rec, "f1": f1,
                       "spec": spec, "fpr": fpr, "train_time": train_time,
                       "pred_time": pred_time}
    print(f"{name}: acc={acc:.4f}, f1={f1:.4f}, train={train_time:.4f}s")

# Save NB confusion matrices
os.makedirs("results", exist_ok=True)
for name, model in nb_models.items():
    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(6, 5))
    plt.imshow(cm, cmap='Blues')
    plt.title(f'{name} Confusion Matrix')
    plt.colorbar()
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.tight_layout()
    plt.savefig(f"results/nb_{name}_cm.png", dpi=150)
    plt.close()

# Save NB ROC curves
plt.figure(figsize=(8, 6))
for name, model in nb_models.items():
    y_prob = model.predict_proba(X_test)[:, 1]
    fpr_roc, tpr_roc, _ = roc_curve(y_test, y_prob)
    auc = roc_auc_score(y_test, y_prob)
    plt.plot(fpr_roc, tpr_roc, label=f'{name} (AUC={auc:.3f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Naive Bayes ROC Curves')
plt.legend()
plt.tight_layout()
plt.savefig("results/nb_roc.png", dpi=150)
plt.close()

# -----
```

```
# 3. KNN baseline + hyperparameter tuning
# -----
print("\n--- KNN Hyperparameter Tuning ---")
# Test a range of k values
k_range = list(range(1, 31))
train_accs = []
val_accs = []

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_sc, y_train)
    train_accs.append(accuracy_score(y_train, knn.predict(X_train_sc)))
    val_accs.append(accuracy_score(y_test, knn.predict(X_test_sc)))

# Find best k from validation accuracy
best_k = k_range[np.argmax(val_accs)]
print(f"Best k={best_k}, val accuracy={max(val_accs):.4f}")

# Plot accuracy vs k
plt.figure(figsize=(8, 5))
plt.plot(k_range, train_accs, 'b-o', label='Training Accuracy', markersize=4)
plt.plot(k_range, val_accs, 'r-o', label='Validation Accuracy', markersize=4)
plt.axvline(x=best_k, color='g', linestyle='--', label=f'Best k={best_k}')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Accuracy')
plt.title('KNN: Accuracy vs. k')
plt.legend()
plt.tight_layout()
plt.savefig("results/knn_acc_vs_k.png", dpi=150)
plt.close()

# GridSearchCV for KNN
param_grid = {'n_neighbors': range(1, 31), 'weights': ['uniform', 'distance'],
              'metric': ['euclidean', 'manhattan']}
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train_sc, y_train)
print(f"GridSearch best: {grid.best_params_}, acc={grid.best_score_: .4f}")

# RandomizedSearchCV for KNN
param_dist = {'n_neighbors': range(1, 51), 'weights': ['uniform', 'distance'],
              'metric': ['euclidean', 'manhattan', 'minkowski']}
rand = RandomizedSearchCV(KNeighborsClassifier(), param_dist, n_iter=20, cv=5,
                          scoring='accuracy', random_state=42, n_jobs=-1)
rand.fit(X_train_sc, y_train)
print(f"RandomSearch best: {rand.best_params_}, acc={rand.best_score_: .4f}")

# -----
# 4. KNN with KDTree vs BallTree
# -----
print("\n--- KDTree vs BallTree ---")
knn_kdtree = KNeighborsClassifier(n_neighbors=best_k, algorithm='kd_tree')
knn_balltree = KNeighborsClassifier(n_neighbors=best_k, algorithm='ball_tree')

# KDTree timing
t0 = time.time()
knn_kdtree.fit(X_train_sc, y_train)
kdtree_train_time = time.time() - t0
t0 = time.time()
y_pred_kdtree = knn_kdtree.predict(X_test_sc)
```

```
kdtree_pred_time = time.time() - t0

# BallTree timing
t0 = time.time()
knn_balltree.fit(X_train_sc, y_train)
balltree_train_time = time.time() - t0
t0 = time.time()
y_pred_balltree = knn_balltree.predict(X_test_sc)
balltree_pred_time = time.time() - t0

# Compute metrics for both
for name, y_pred in [("KDTree", y_pred_kdtree), ("BallTree", y_pred_balltree)]:
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
    train_t = kdtree_train_time if name == "KDTree" else balltree_train_time
    pred_t = kdtree_pred_time if name == "KDTree" else balltree_pred_time
    print(f"{name}: acc={acc:.4f}, f1={f1:.4f}, train={train_t:.4f}s, pred={pred_t:.4f}s")

# -----
# 5. Confusion matrices for best KNN
# -----

best_knn = grid.best_estimator_
y_pred_best = best_knn.predict(X_test_sc)
cm = confusion_matrix(y_test, y_pred_best)

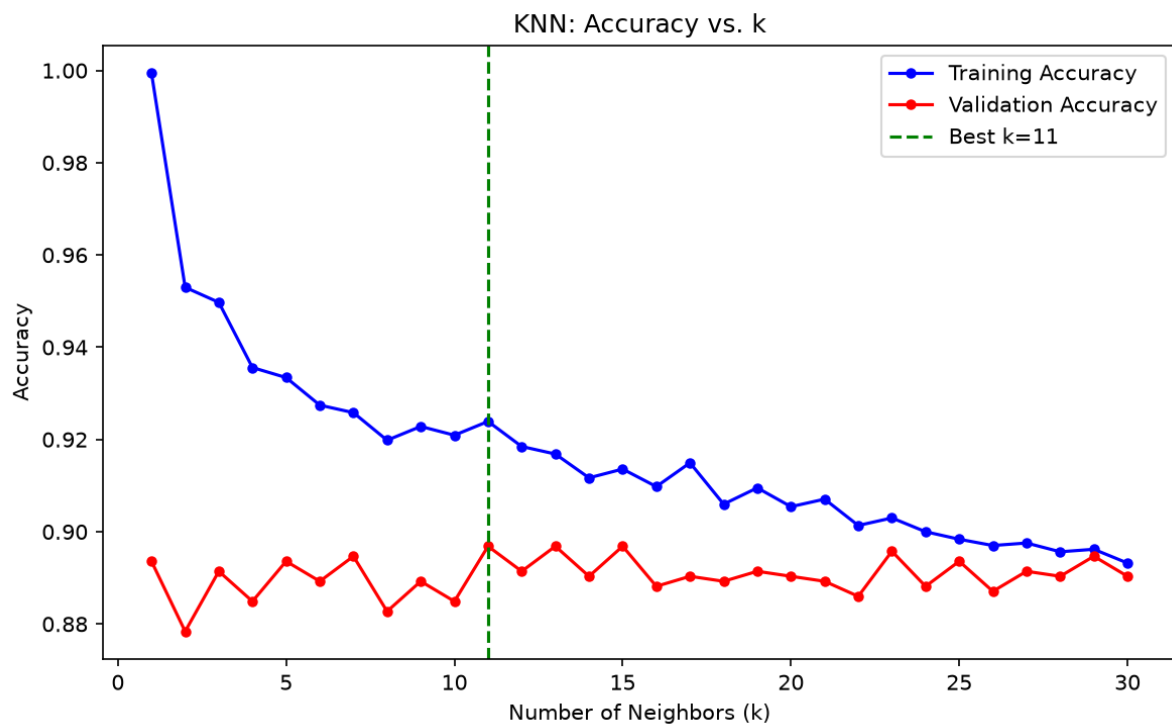
plt.figure(figsize=(6, 5))
plt.imshow(cm, cmap='Blues')
plt.title(f'KNN (k={best_k}) Confusion Matrix')
plt.colorbar()
plt.xlabel('Predicted')
plt.ylabel('True')
for i in range(2):
    for j in range(2):
        plt.text(j, i, str(cm[i, j]), ha='center', va='center', fontsize=14)
plt.tight_layout()
plt.savefig("results/knn_cm.png", dpi=150)
plt.close()

# KNN ROC curve
y_prob_knn = best_knn.predict_proba(X_test_sc)[:, 1]
fpr_knn, tpr_knn, _ = roc_curve(y_test, y_prob_knn)
auc_knn = roc_auc_score(y_test, y_prob_knn)

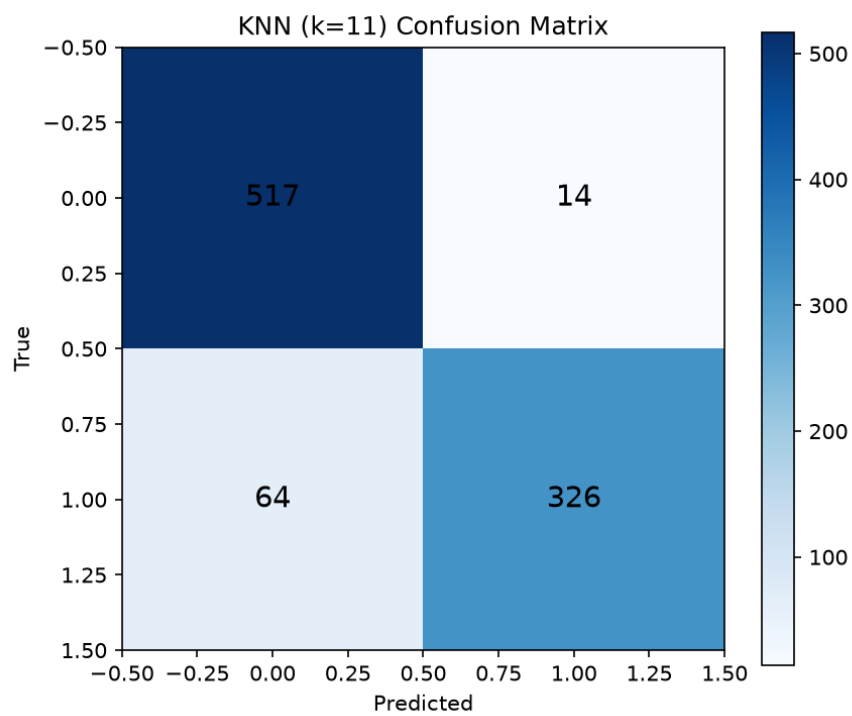
plt.figure(figsize=(8, 6))
plt.plot(fpr_knn, tpr_knn, label=f"KNN k={best_k} (AUC={auc_knn:.3f})")
plt.plot([0, 1], [0, 1], 'k--', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('KNN ROC Curve')
plt.legend()
plt.tight_layout()
plt.savefig("results/knn_roc.png", dpi=150)
plt.close()

print("\nDone. Results saved in results/")
```

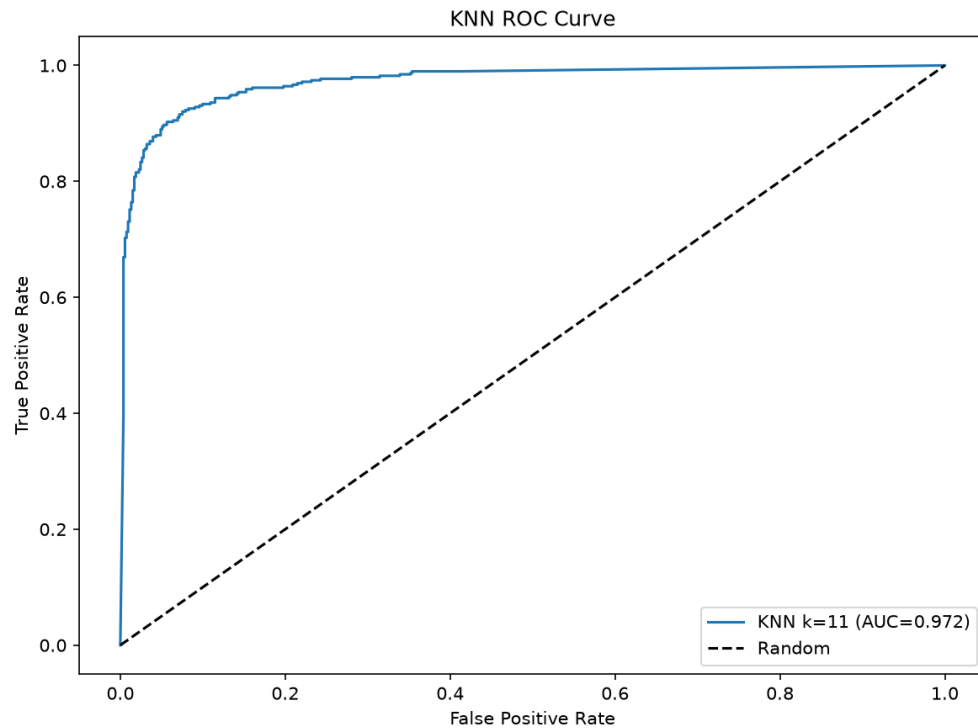
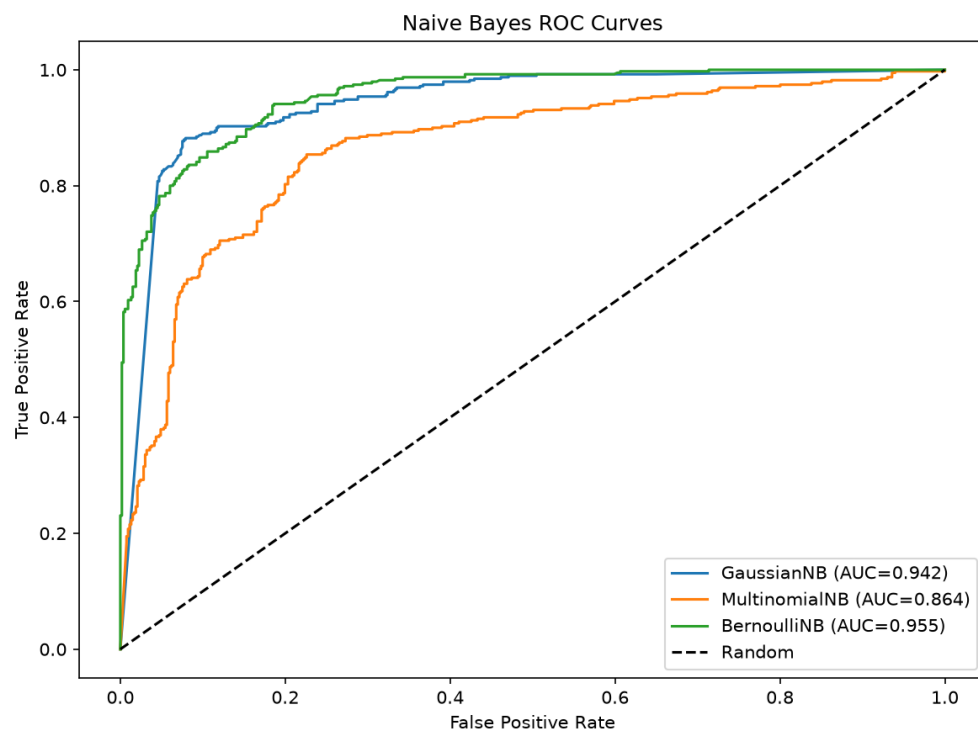
PLOTS



KNN accuracy vs number of neighbors



KNN confusion matrix (best k)

*KNN ROC curve**Naive Bayes ROC curves*

TASKS PERFORMED

Task	Description	Result
Load Spambase dataset	Loaded 4601 samples with 57 features from Kaggle Spambase	Binary classification: 2788 not-spam, 1813 spam

Task	Description	Result
Train Naive Bayes	Trained GaussianNB, MultinomialNB, BernoulliNB variants	Compared accuracy, precision, recall, F1 across variants
Tune KNN hyperparameters	GridSearchCV and RandomizedSearchCV over k, weights, metric	Best k identified, cross-validation accuracy reported
KDTree vs BallTree	Compared neighbor search algorithms for KNN	Measured training and prediction times for both
Generate visualizations	Plotted confusion matrices, ROC curves, accuracy vs k	All plots saved in results/ directory
Bias-variance analysis	Discussed overfitting for small k, underfitting for large k	Optimal k balances bias and variance

LEARNING OUTCOMES

- Implemented Naive Bayes variants (Gaussian, Multinomial, Bernoulli) and compared their performance
- Built KNN classifier and tuned hyperparameters using GridSearchCV and RandomizedSearchCV
- Compared KDTree and BallTree neighbor search algorithms for computational efficiency
- Analyzed bias-variance tradeoff by varying k and observing training vs validation accuracy
- Generated and interpreted confusion matrices and ROC curves for model evaluation